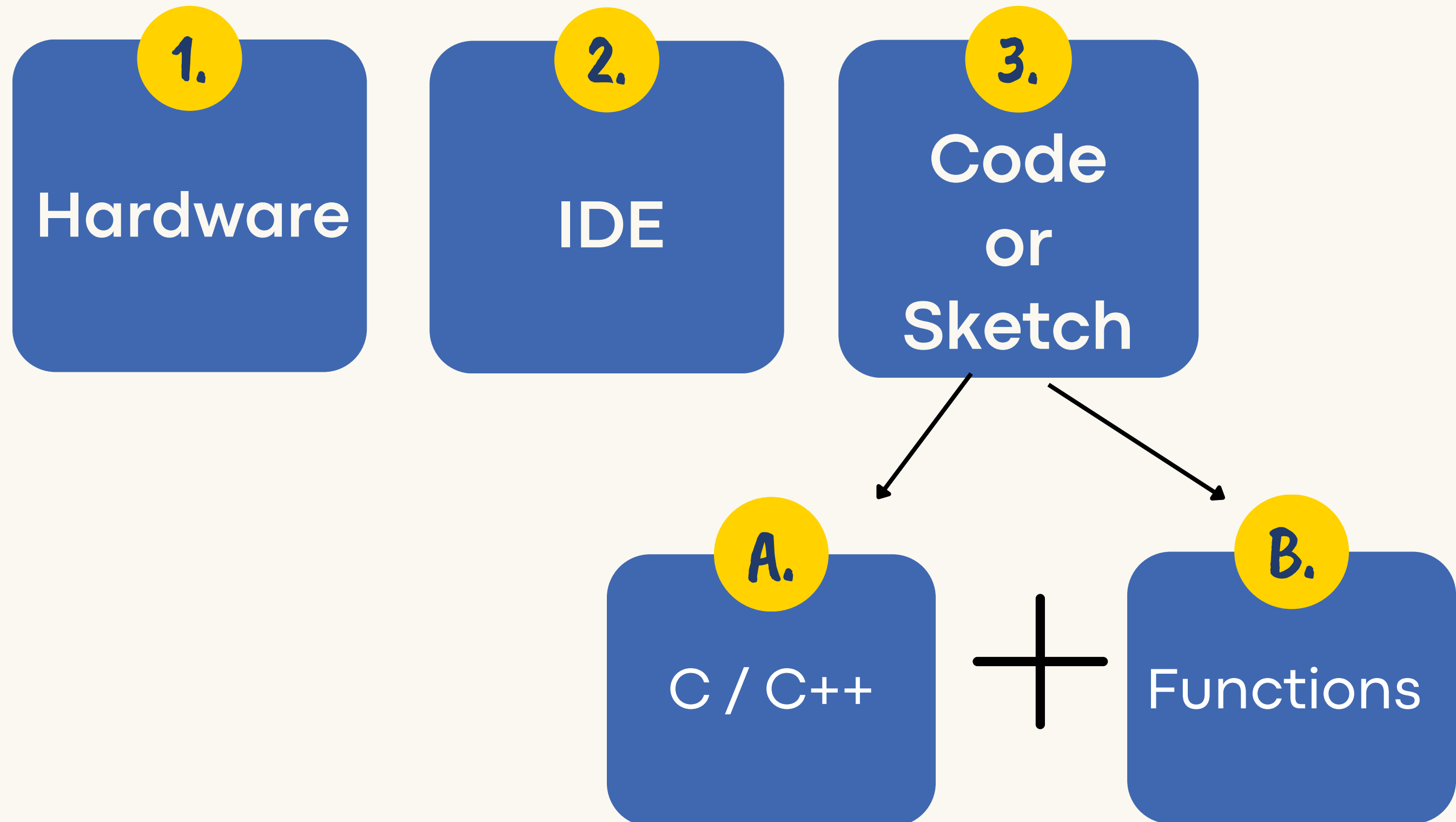


Arduino Programming



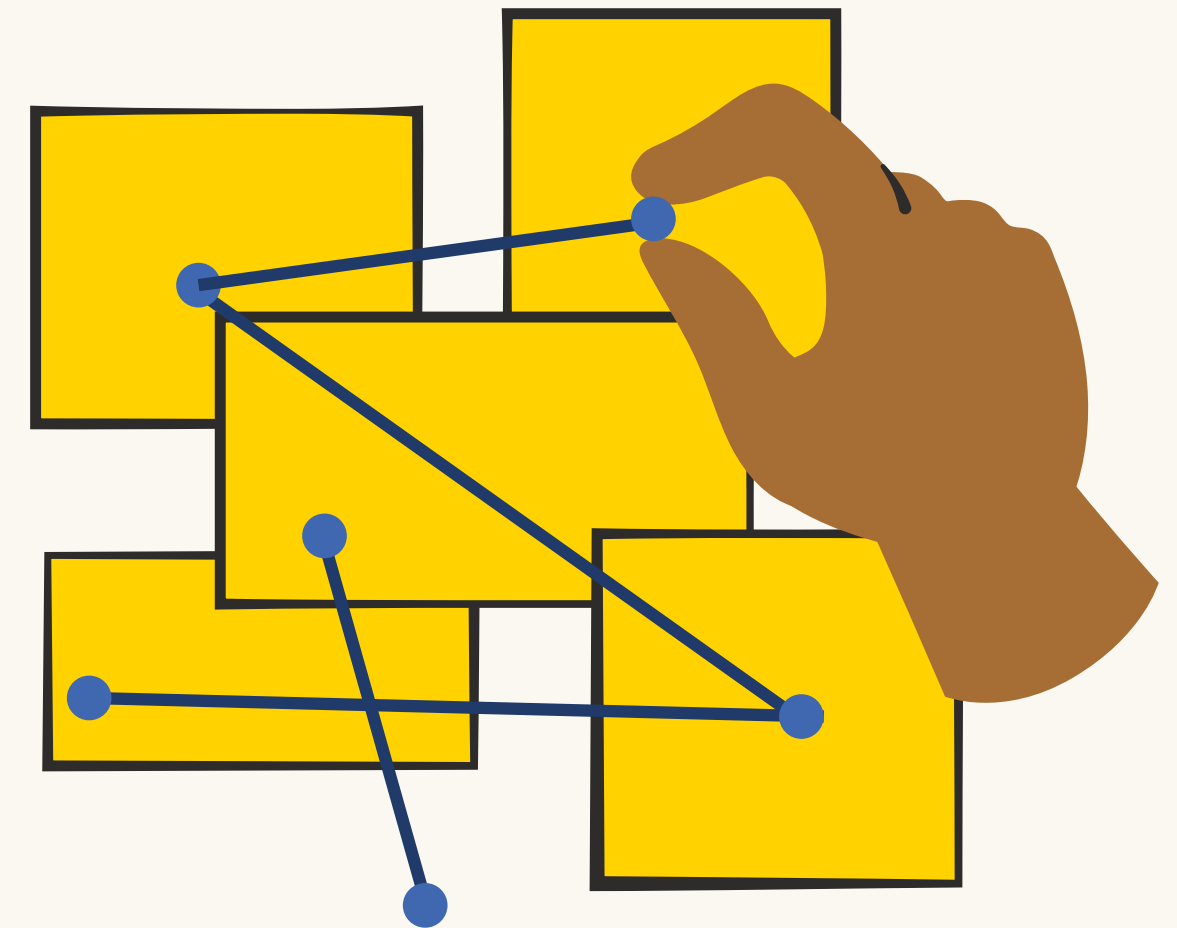
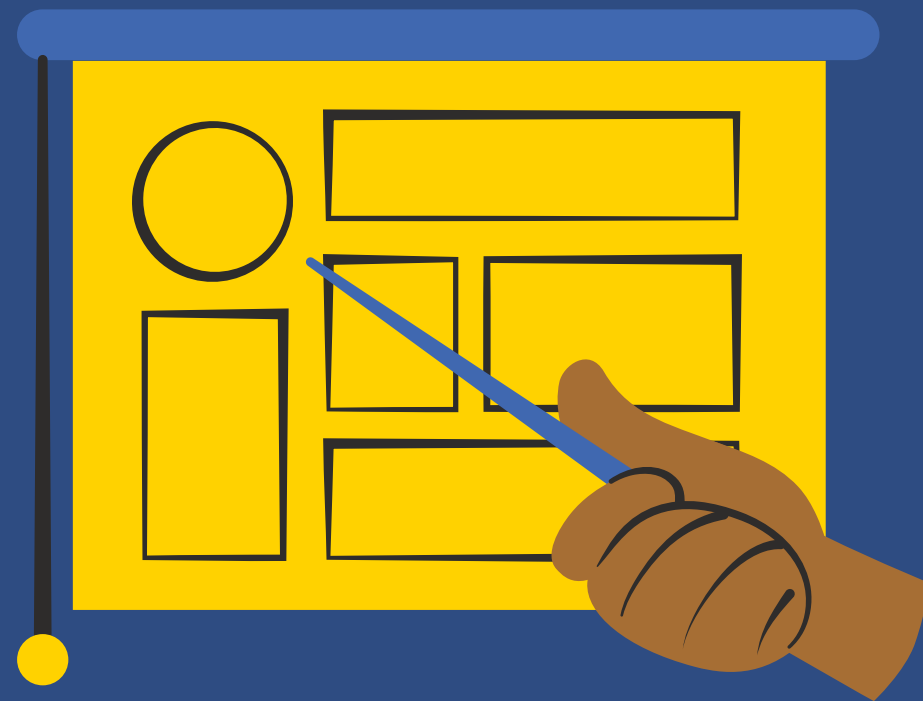
Today, you'll learn about

Arduino Trifecta



Setup Function

This function is called once when the microcontroller is powered up or reset. It is used for initializing variables, pin modes, libraries, etc.



Syntax:
void setup()
{
}

Example

```
void setup()  
{  
  pinMode(8, OUTPUT); // Initialize pin 8 as output  
}
```



Anything that needs to be done once at the beginning of the program should go in this function.

pinMode Function

It is a function used to configure a specific digital pin on the microcontroller as either an input or an output. This function is used within the `setup()` function to initialize the mode of a pin before using it.



Syntax: `pinMode(PinNumber, Mode)`

Parameters

Pin Number

specifies the number of the digital pin you want to configure

Mode

- INPUT
- OUTPUT
- INPUT_PULLUP

Example

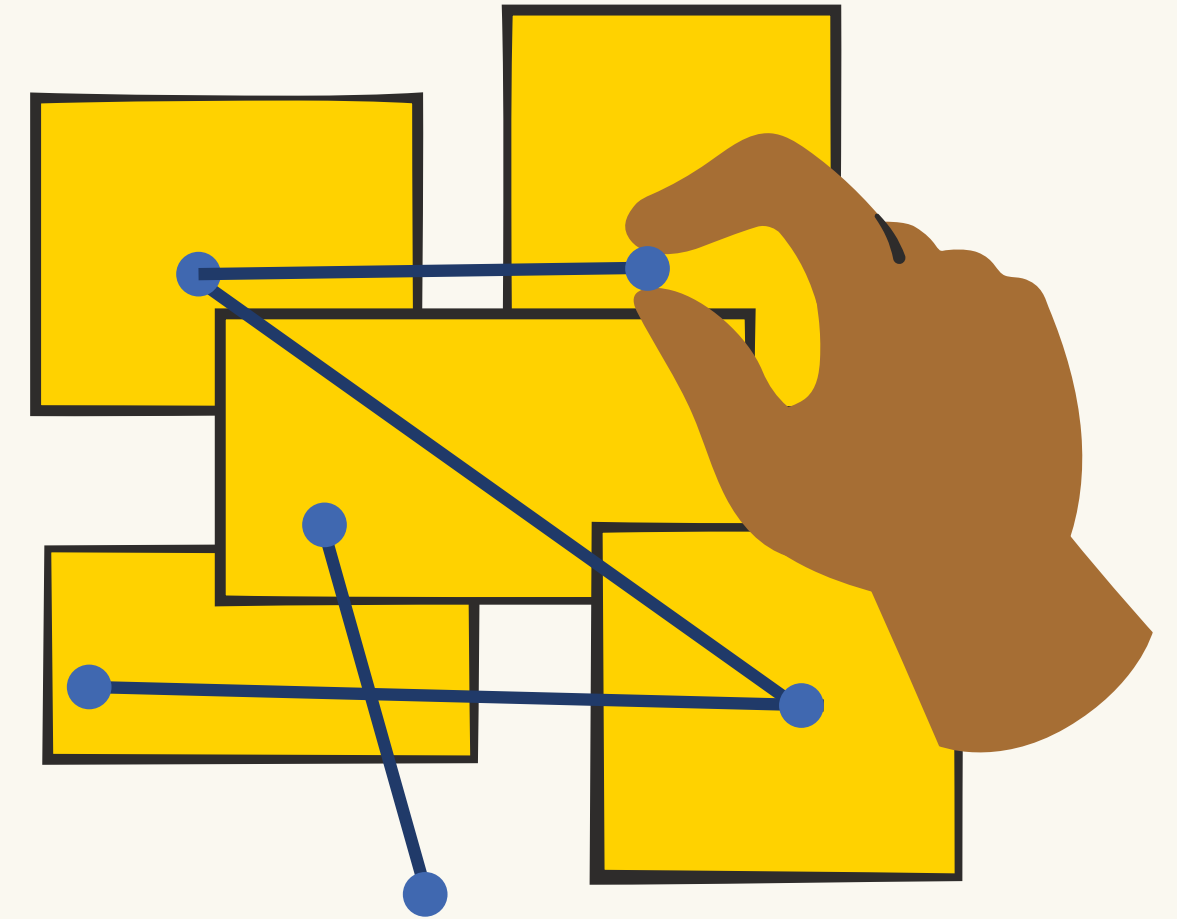
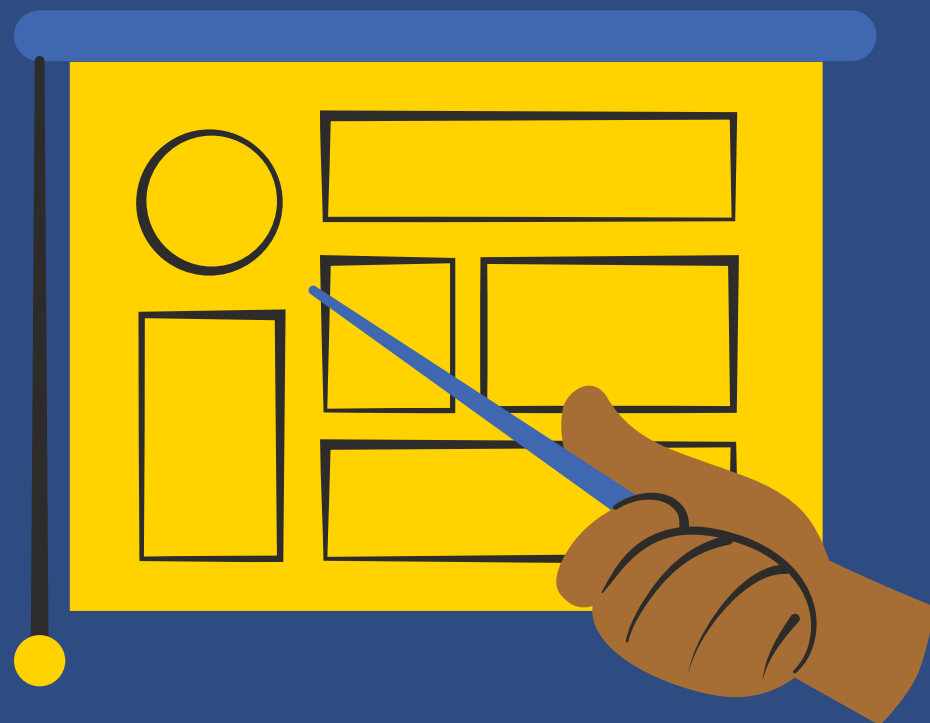
```
void setup() {  
  pinMode(8, OUTPUT); // Configure digital  
  pin 8 as an output  
  pinMode(7, INPUT); // Configure digital  
  pin 7 as an input  
}
```



There are 14 digital and 6 analog pins

Loop Function

This function runs repeatedly as long as the microcontroller is powered up. It contains the main code logic and is where you put the instructions for your project's behavior. The code inside this function will execute in a loop until the microcontroller loses power or is reset.



Syntax:
`void loop()
{
}
}`

Example

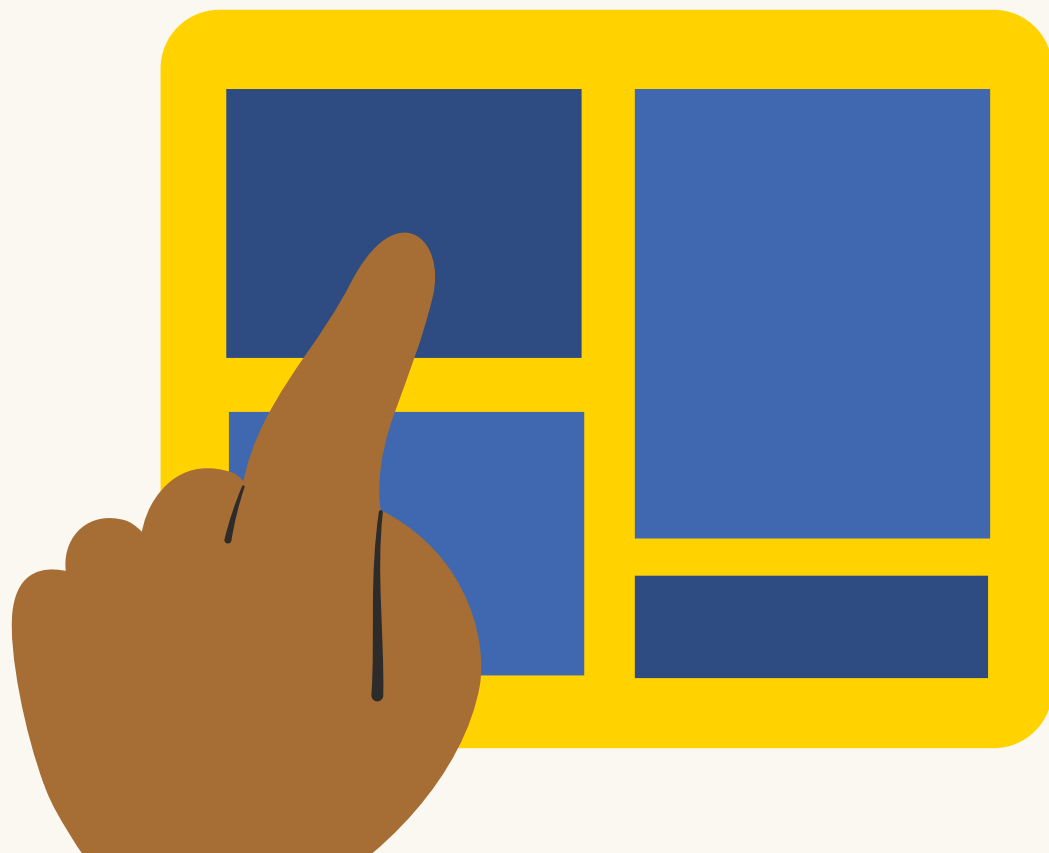
```
void loop() {  
  digitalWrite(9, HIGH); // Turn on  
  delay(1000); // Wait for 1 second  
  digitalWrite(9, LOW); // Turn off  
  delay(1000); // Wait for 1 second  
}
```



Anything that needs to be repeated in the program should go in this function.

digitalWrite Function

It is used to set the digital state (voltage level) of a specified digital pin on the Arduino board. It allows you to control external devices such as LEDs, relays, or other digital components that can be either ON or OFF.



Syntax: `digitalWrite(pin, value);`

Parameters

Pin

specifies the number of the digital pin you want to configure

value

- HIGH
- LOW

Example

```
// Set digital pin 13 to HIGH (5V)  
digitalWrite(13, HIGH);
```

```
// Set digital pin 8 to LOW (0V)  
digitalWrite(8, LOW);
```



There are 14 digital pins

Example: Controlling LEDs:

```
void setup() {  
  pinMode(13, OUTPUT);  
}  
  
void loop() {  
  digitalWrite(13, HIGH); // Turn on the LED  
  delay(1000);             // Wait for 1 second  
  digitalWrite(13, LOW);  // Turn off the LED  
  delay(1000);             // Wait for 1 second  
}
```



Example: Driving Relays:

```
int relayPin = 7;
void setup() {
  pinMode(relayPin, OUTPUT);
}
void loop() {
  digitalWrite(relayPin, HIGH);
  delay(1000);           // Wait for 1 second
  digitalWrite(relayPin, LOW);
  delay(1000);           // Wait for 1 second
}
```



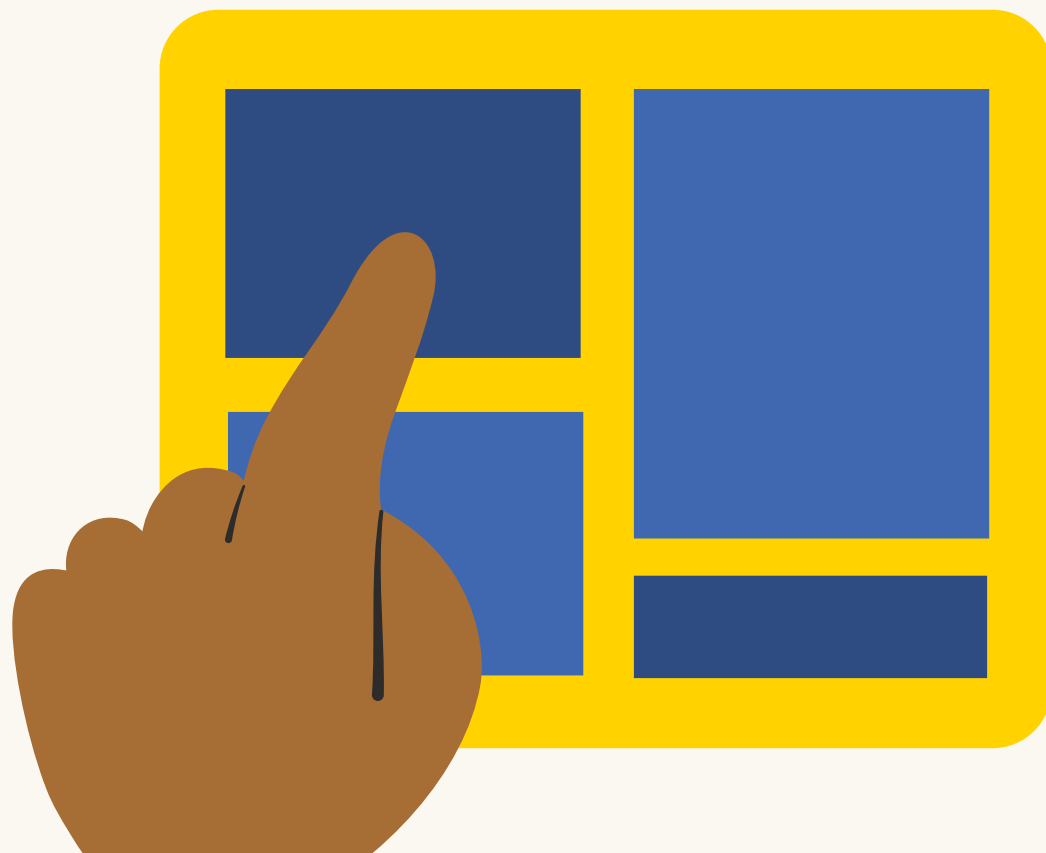
Example: Digital Components

```
int digitalComponentPin = 3;
void setup() {
  pinMode(digitalComponentPin, OUTPUT);
}
void loop() {
  digitalWrite(digitalComponentPin, HIGH);
  delay(500);           // Wait for 0.5 seconds
  digitalWrite(digitalComponentPin, LOW); //
  Turn OFF the digital component
  delay(500);           // Wait for 0.5 seconds
}
```



digitalRead Function

is used to read the digital state of a specified digital pin. This function is commonly employed when we want to interact with digital sensors, switches, buttons, or any other devices that produce a digital HIGH or LOW signal.



Syntax: `int digitalRead(int pin);`

Parameters

Pin

The pin number from which to read the digital state. This can be a specific digital pin number (e.g., 2, 7, 13) or a constant representing a predefined pin (e.g., A0 for analog pin 0).

digitalRead Function

Return Value:

The function returns an integer value:

- **HIGH (1):** If the digital voltage at the specified pin is at a high level (typically 5V).
- **LOW (0):** If the digital voltage at the specified pin is at a low level (typically 0V or close to it).



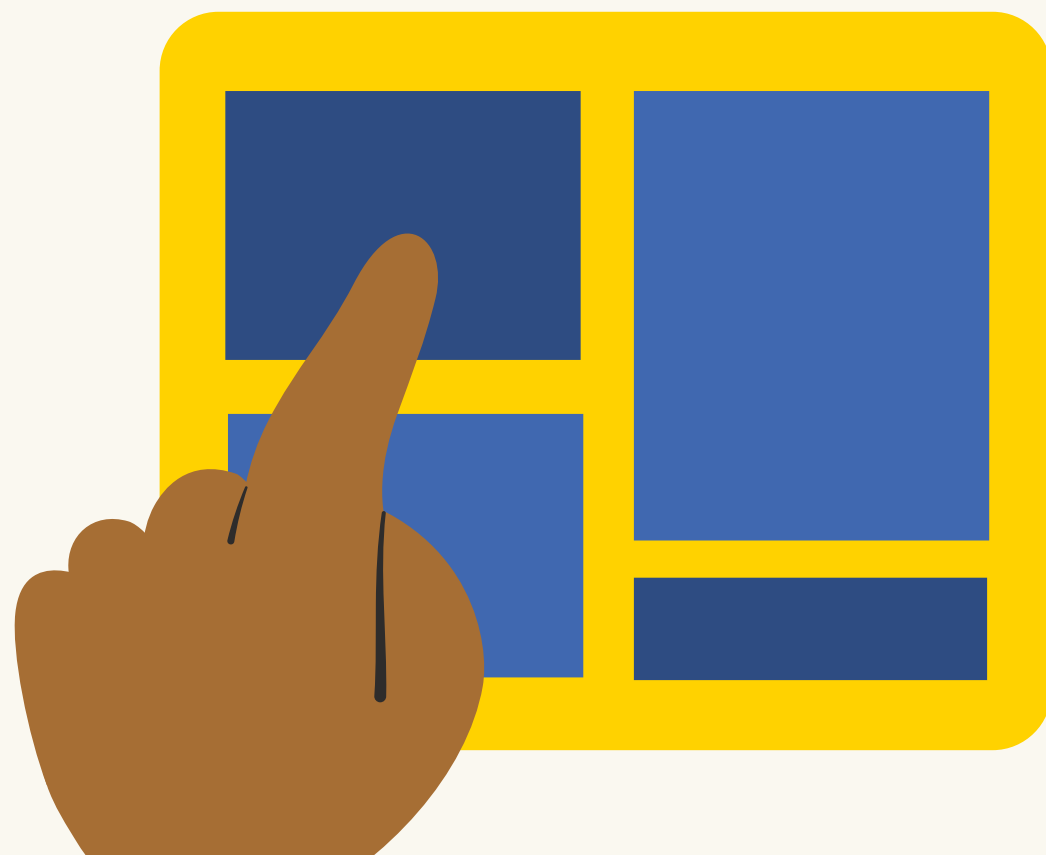
Example

```
int buttonPin = 2; // Connect a button to digital
pin 2
int buttonState;
void setup() {
  pinMode(buttonPin, INPUT);
}
void loop() {
  buttonState = digitalRead(buttonPin);
  if (buttonState == HIGH) {
    Serial.println("Button pressed!");
  } else {
    Serial.println("Button not pressed."); } }
```



analogRead Function

It is used to read the analog voltage present on a specified analog pin. The Arduino Uno has several analog pins marked as A0 through A5. Analog pins can read a range of voltages.



Syntax: `int analogRead(int pin);`

Parameters

Pin

The analog pin number from which to read the analog voltage. For example, A0, A1, ..., A5

analogRead Function

Return Value:

The function returns an integer value between 0 and 1023, representing the analog voltage level on the specified pin. 0 corresponds to 0V, and 1023 corresponds to the reference voltage (usually 5V on most Arduino boards).



Example

```
int sensorValue = analogRead(A0);
```



There are 6 Analog pins

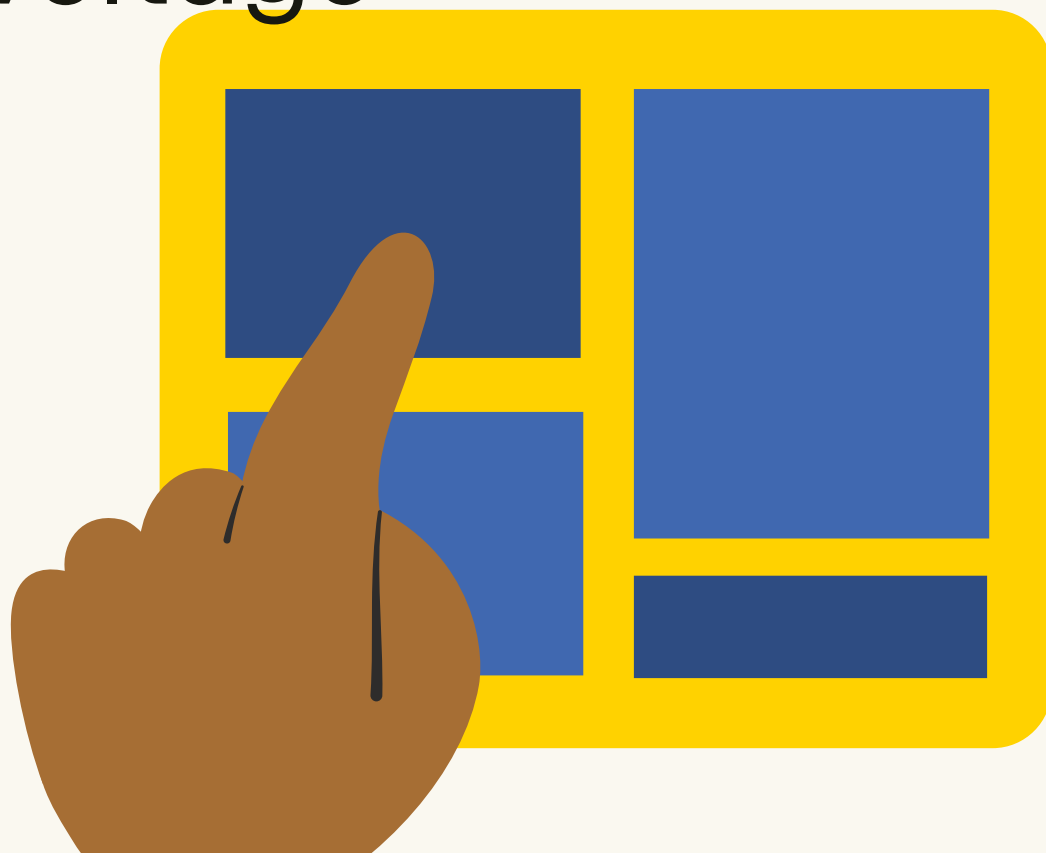
```
const int potentiometerPin = A1;
void setup() {
  Serial.begin(9600); // Initialize serial communication
}
void loop() {
  int potValue = analogRead(potentiometerPin);
  float voltage = potValue * (5.0 / 1023.0); // Convert to
voltage (0V to 5V)
  Serial.print("Potentiometer Value: ");
  Serial.print(potValue);
  Serial.print(", Voltage: ");
  Serial.println(voltage);
  delay(500); // Add a short delay for stability
}
```



analogWrite Function

Syntax: `analogWrite (pin, value);`

It is used to generate a Pulse Width Modulation (PWM) signal on a specified digital pin. PWM is a technique where the average value of a pulsating signal is used to represent an analog voltage



Parameters

Pin

The digital pin to which the PWM signal will be sent

value

It is an integer value between 0 and 255

Example

```
analogWrite(9, 128); // Send a PWM  
signal with a duty cycle of 50% to  
digital pin 9
```

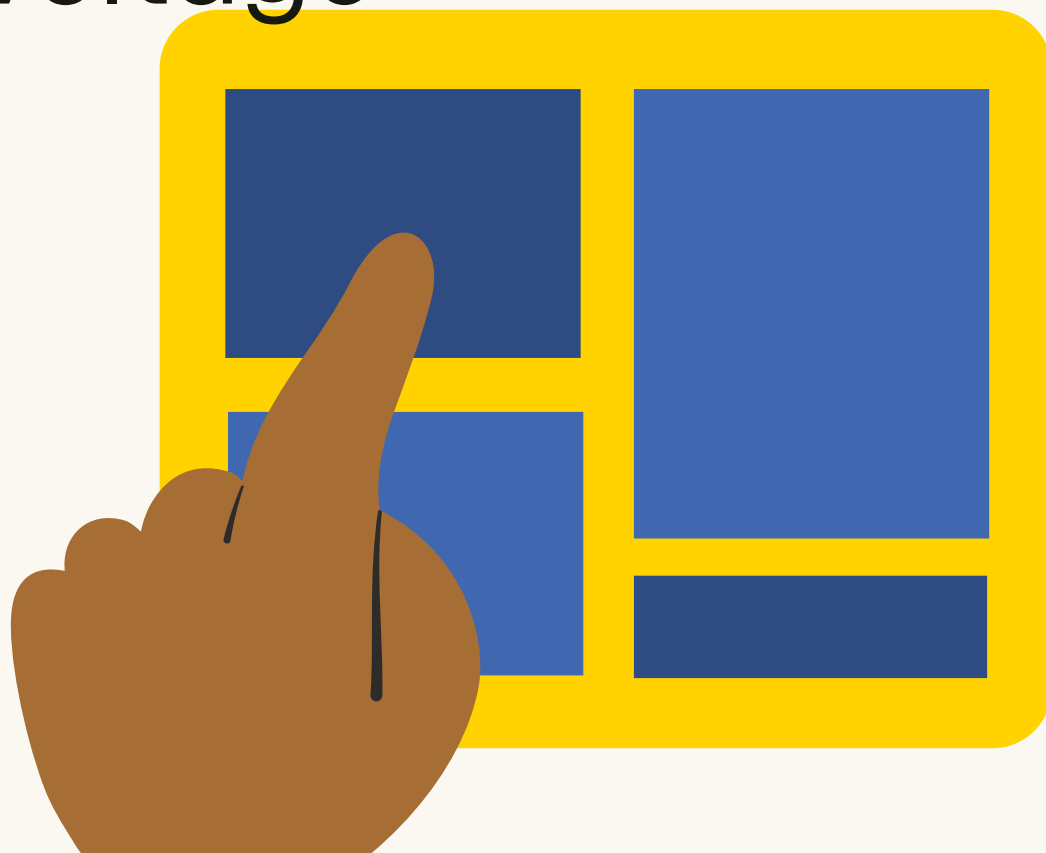


Digital pins are used to send analog signal

analogWrite Function

Syntax: `analogWrite (pin, value);`

It is used to generate a Pulse Width Modulation (PWM) signal on a specified digital pin. PWM is a technique where the average value of a pulsating signal is used to represent an analog voltage



Parameters

Pin

The digital pin to which the PWM signal will be sent

value

It is an integer value between 0 and 255

Example

```
analogWrite(9, 128); // Send a PWM  
signal with a duty cycle of 50% to  
digital pin 9
```



Digital pins are used to send analog signal

```
const int ledPin = 9; // LED connected to digital pin 9
void setup() {
  pinMode(ledPin, OUTPUT); }
void loop() {
  for (int brightness = 0; brightness <= 255; brightness +=
5) {
    analogWrite(ledPin, brightness); // Set LED brightness
    delay(50); // Delay for a smooth transition
  }
  for (int brightness = 255; brightness >= 0; brightness -=
5) {
    analogWrite(ledPin, brightness); // Set LED brightness
    delay(50); // Delay for a smooth transition
  } }
```



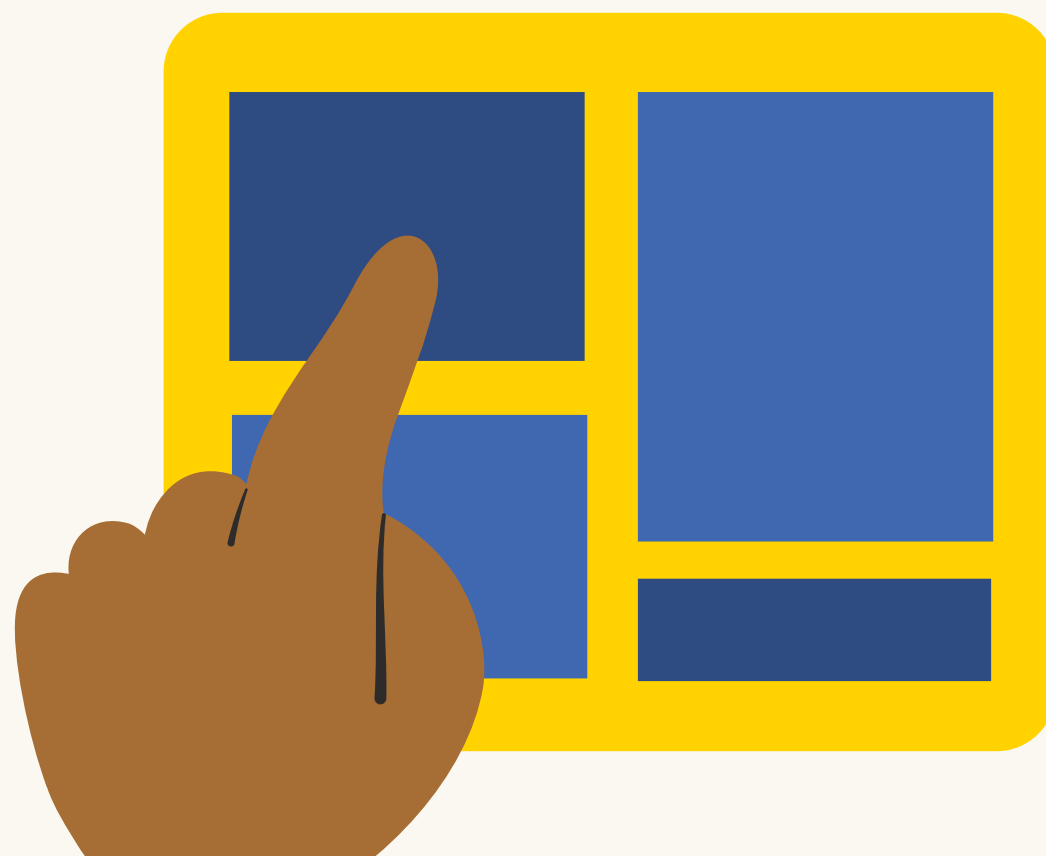
Advanced I/O functions

Tone Generation



Tone Function

Generates a tone of a specified frequency on a specified pin



Syntax: `tone (pin, frequency);`

Parameters

Pin

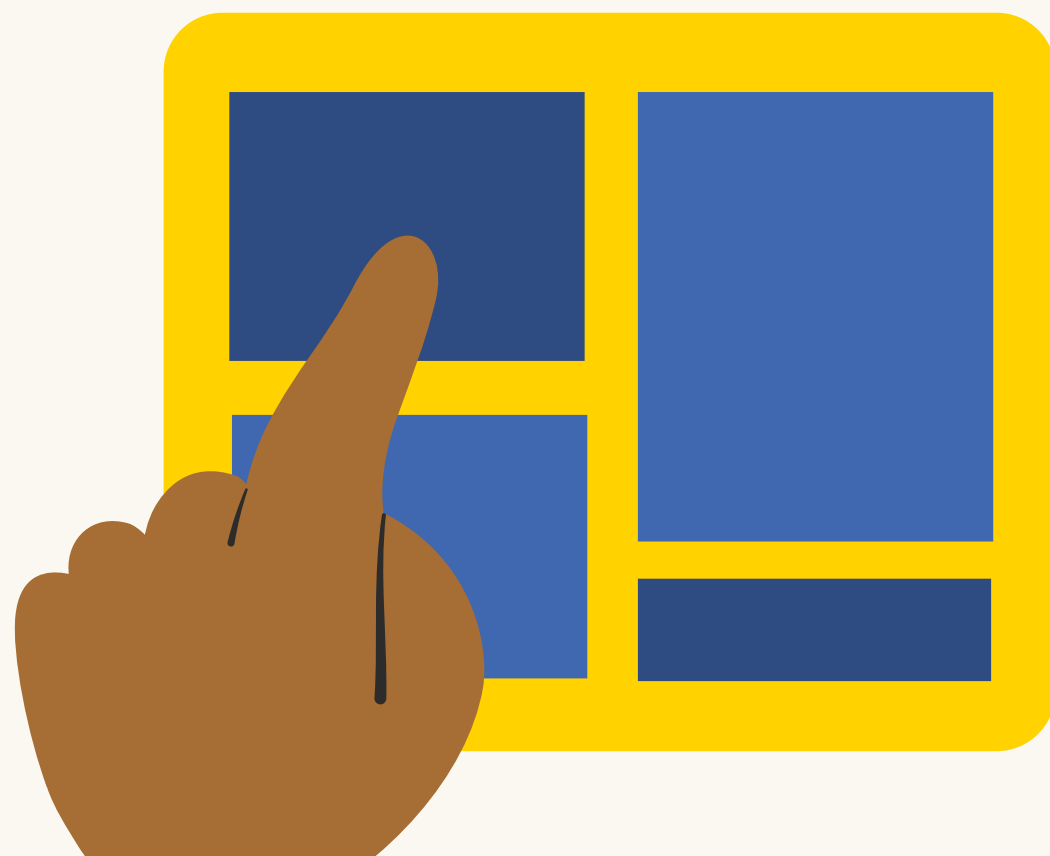
The digital pin to which the piezo buzzer or speaker is connected

frequency

The frequency of the tone in hertz (Hz). It determines the pitch of the sound.

Tone Function

Generates a tone of a specified frequency on a specified pin for a specified duration in milliseconds



Syntax: `tone (pin, frequency, duration);`

Parameters

Pin

The digital pin to which the piezo buzzer or speaker is connected

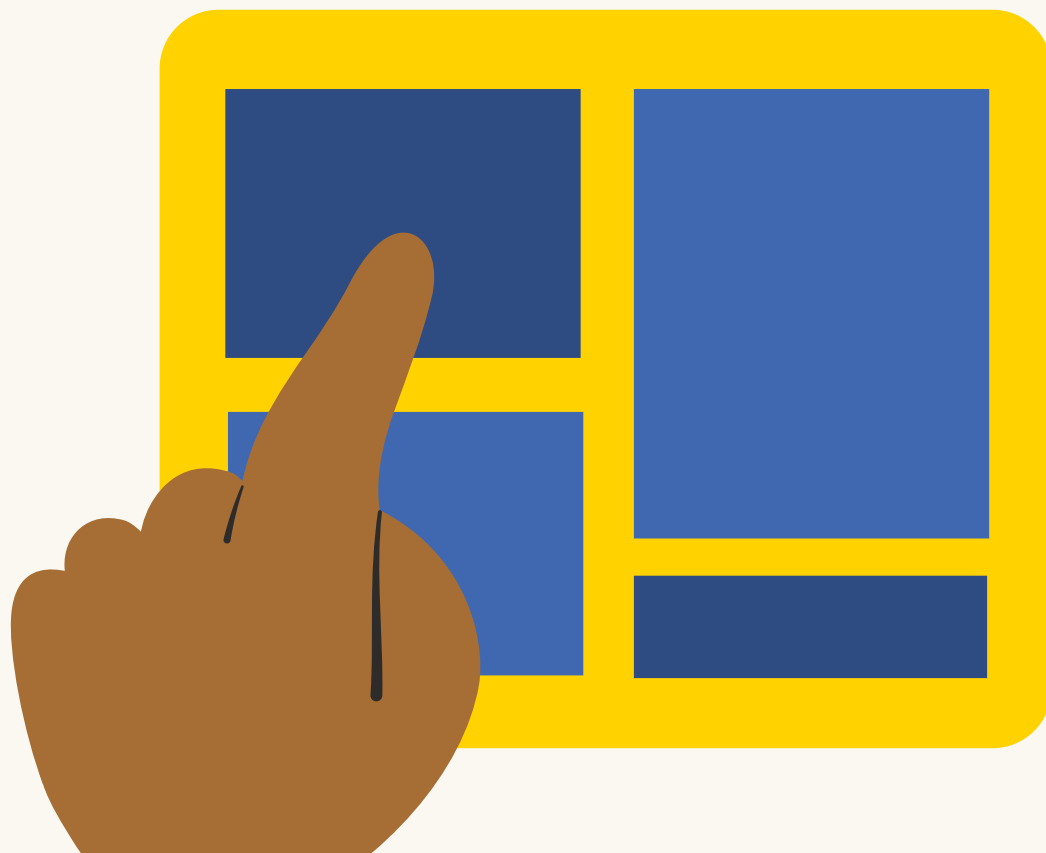
frequency

The frequency of the tone in hertz (Hz). It determines the pitch of the sound.

duration

noTone Function

Stops the tone on the specified pin



Syntax: `noTone (pin);`

Parameters

Pin

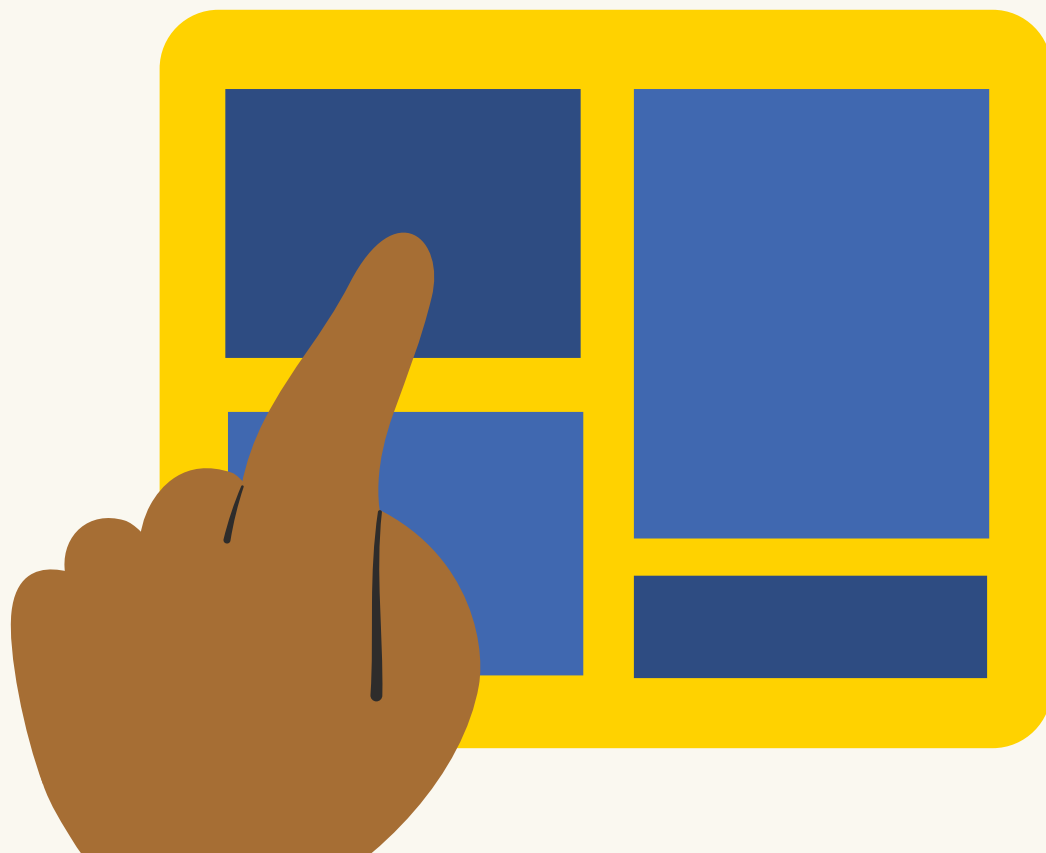
The digital pin to which the piezo buzzer or speaker is connected

```
const int buzzerPin = 8;
void setup() {
  // No setup is required for this example
}
void loop() {
  // Generate a tone of 1000Hz for 500 milliseconds
  tone(buzzerPin, 1000, 500);
  delay(500);
  noTone(buzzerPin);
  delay(500);
}
```



pulseIn Function

Measures the duration of a pulse on a specified digital pin and returns the duration in microsecond



Syntax: *pulseIn* (pin, state);

Parameters

Pin

The digital pin on which to measure the pulse duration.

state

The type of pulse to measure, either HIGH or LOW

pulseIn Function

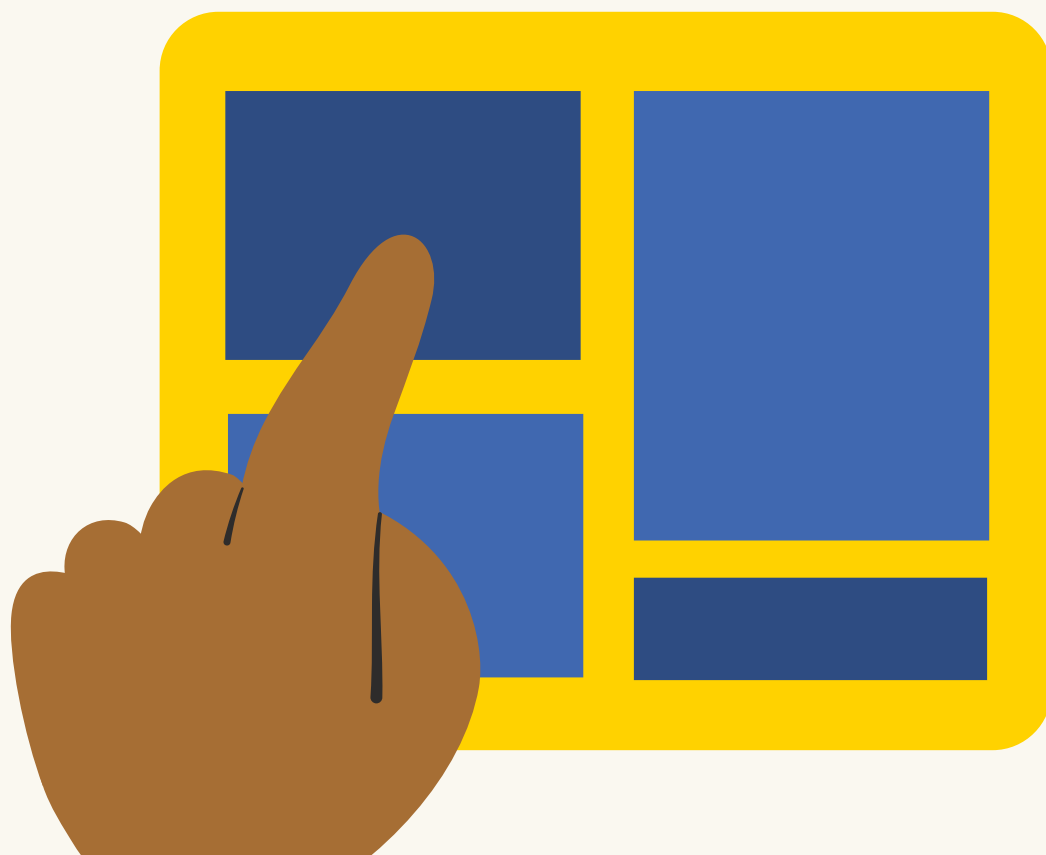
Return Value:

The function returns the duration of the pulse in microseconds as an unsigned int data type



pulseInLong Function

Measures the duration of a pulse on a specified digital pin and returns the duration in microsecond



Syntax: *pulseInLong* (pin, state);

Parameters

Pin

The digital pin on which to measure the pulse duration.

state

The type of pulse to measure, either HIGH or LOW

pulseInLong Function

Return Value:

*The function returns the duration of the pulse in
microseconds as an unsigned long data types*



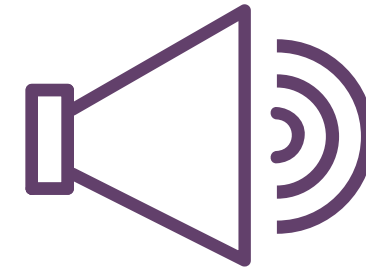
```
unsigned int duration;  
unsigned long lduration;  
void setup() {  
  Serial.begin(9600);  
  pinMode(2, INPUT);  
  pinMode(3, INPUT);  
}  
void loop() {  
  duration = pulseIn(2, HIGH);  
  lduration = pulseInlong(3, LOW);  
}
```



Timer Functions



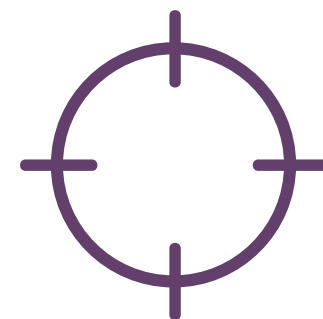
`delay ()`



`delayMicrosec ()`



`millis ()`



`micros ()`

delay Function

Pauses the Program execution for the specified number of milliseconds

Syntax

delay (ms)

Parameter

ms - milliseconds

delayMicroseconds Function

Pauses the Program
execution for the
specified number of
microseconds

Syntax

delayMicroseconds (us)

Parameter

us - microseconds

Example: Controlling LEDs:

```
void setup() {  
  pinMode(13, OUTPUT);  
}  
void loop() {  
  digitalWrite(13, HIGH); // Turn on the LED  
  delay(100);             // Wait for 100 milliseconds  
  digitalWrite(13, LOW);  // Turn off the LED  
  delayMicroseconds(100); // Wait for 100  
  microsecond  
}
```



millis Function

Returns the number of milliseconds passed since the Arduino board began running the current program

Syntax

`millis ( )`

Note

This number will overflow (go back to 0) after approx 50 days

micros Function

Returns the number of microseconds passed since the Arduino board began running the current program

Syntax

`micros ( )`

Note

This number will overflow (go back to 0) after approx 70 minutes

Example: Controlling LEDs:



```
unsigned long time
void setup() {
  Serial.begin(9600);
}
void loop() {
  Serial.print("Time: ");
  time = millis ();
  Serial.println(time);
  delay (1000);
}
```

Example: Controlling LEDs:

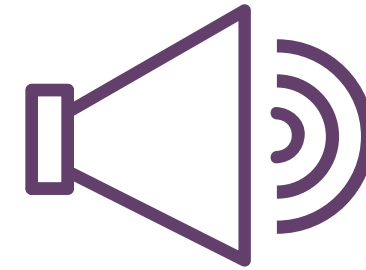
```
unsigned long time
void setup() {
  Serial.begin(9600);
}
void loop() {
  Serial.print("Time: ");
  time = micros();
  Serial.println(time);
  delay (1000);
}
```



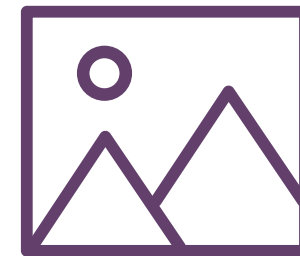
Communication Functions



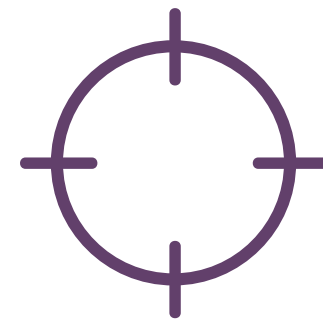
Serial Communication



Serial Peripheral
Interface



Stream



I2C

SERIAL COMMUNICATION (UART)

Functions

Serial.begin(baudRate)

Initializes serial communication with a specific baud rate

Serial.available()

Returns the number of bytes available to read from the serial port

Serial.read()

Reads a byte of data from the serial port

SERIAL COMMUNICATION (UART)

Functions

Serial.write(data)

Sends a byte of
data to the serial
port

Serial.print(data)

Sends data to the
serial port

Serial.println(data)

Sends data to the
serial port
followed by a
newline character

SPI



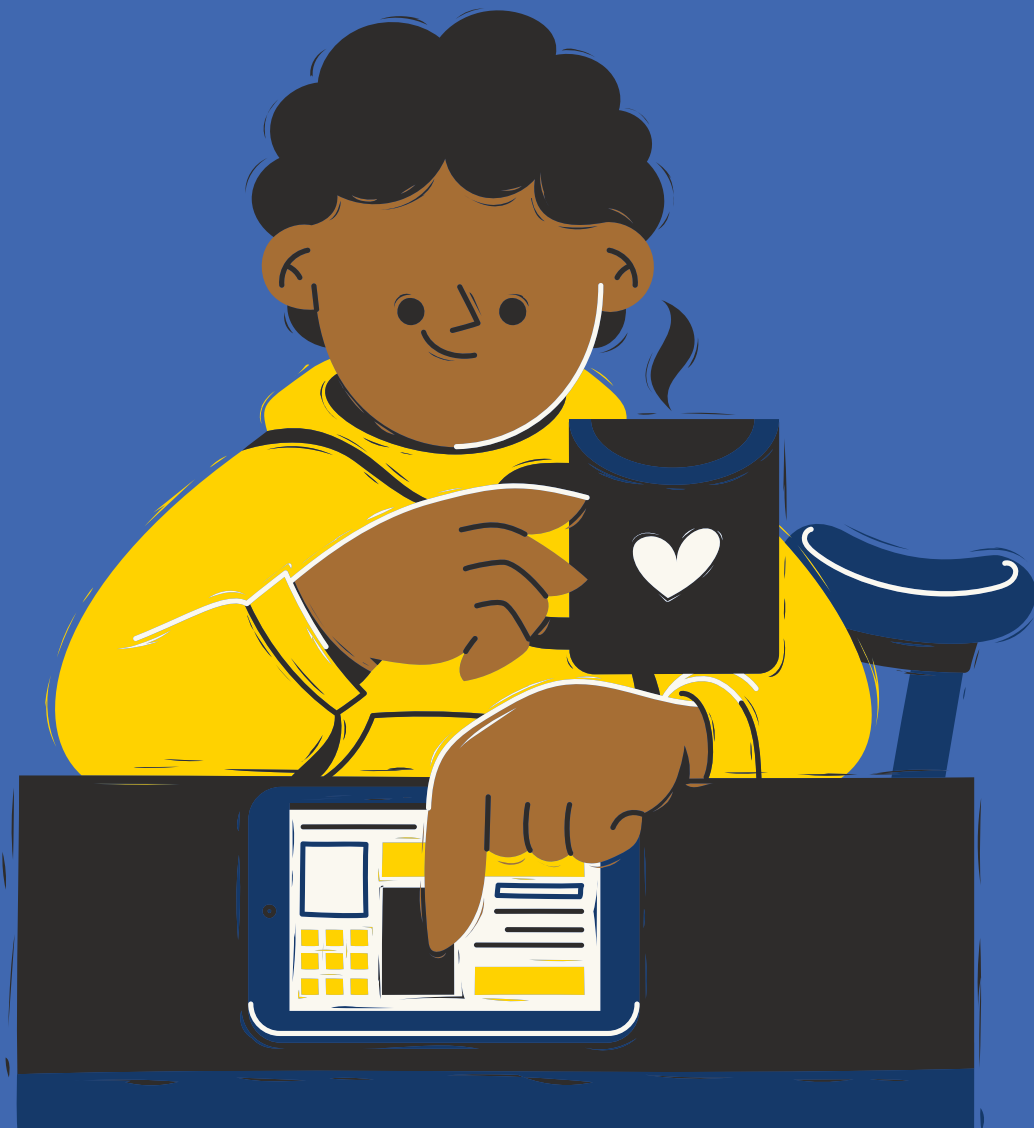
01

Used for high-speed data transfer

02

```
#include<SPI.h>
```

FUNCTIONS



SPI.begin ()

Initializes the SPI communication

SPI.transfer(data)

Sends and receives data

SPI.endTransaction

Ends the SPI transaction

SPI.beginTransaction()

```
SPI.beginTransaction  
(SPISettings(clock,  
bitOrder, dataMode))
```



What is the main idea?

Starts a transaction with specific settings including clock speed, bit order and data mode

STREAM

Functions

read()

Reads a byte of
data

write(data)

Writes data

available ()

Checks how many
bytes are available to
read

STREAM

Functions

`readBytes(buffer, length)`

Reads a specified number of bytes into a buffer

`readBytesUntil(terminatorbuffer, length)`

Reads bytes until a specified terminator character into a buffer

I2C



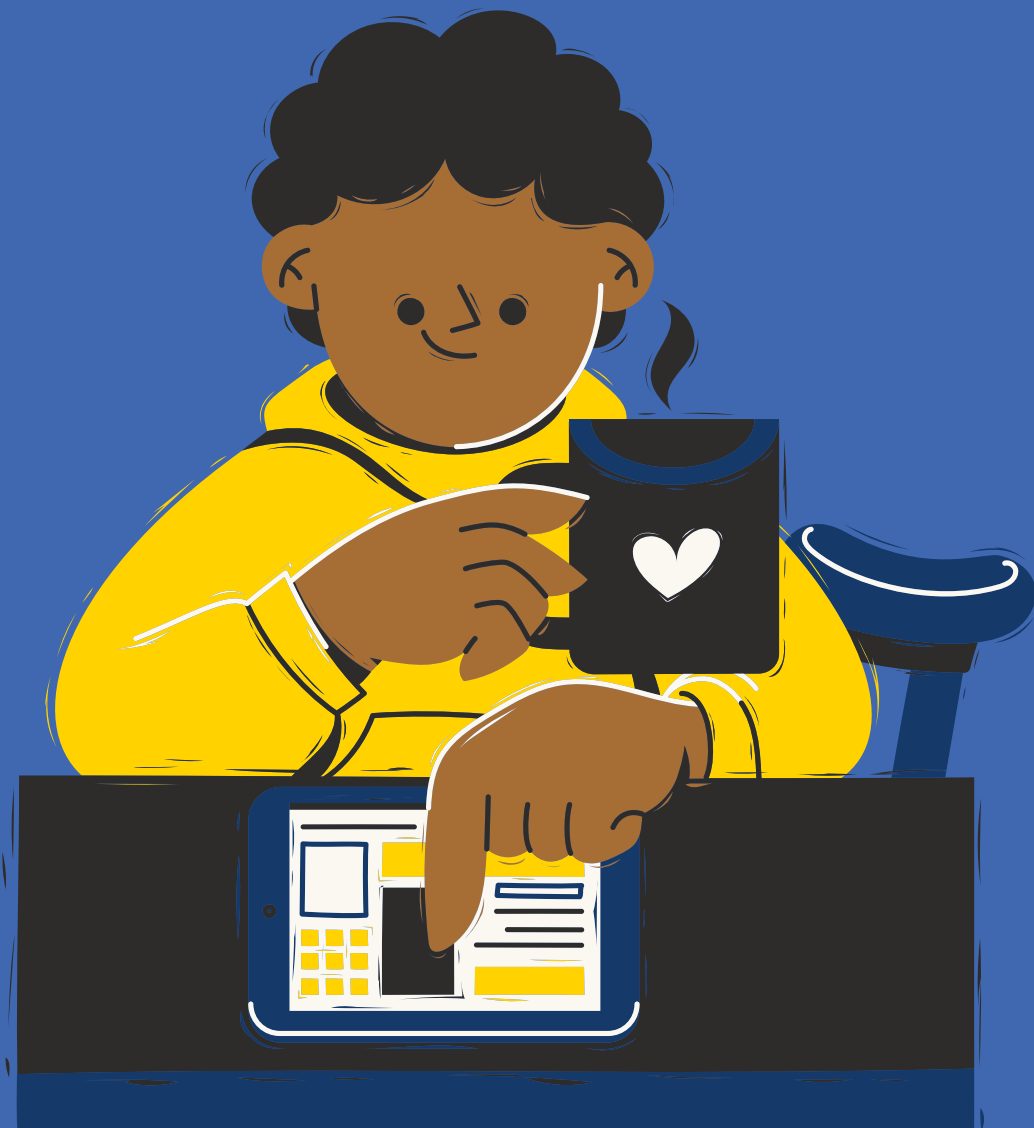
01

**Multimaster, multi-slave, 2 wire
serial communication**

02

```
#include<Wire.h>
```

FUNCTIONS



Wire.begin ()

Initializes the I2C communication protocol

Wire.write(data)

Sends data

Wire.endTransmission()

Ends the I2C transaction

Wire.beginTransaction()

Wire.beginTransaction(ad
dress)



What is the main idea?

**Begins a transmission to a
specified I2C address**

Wire.requestFrom()

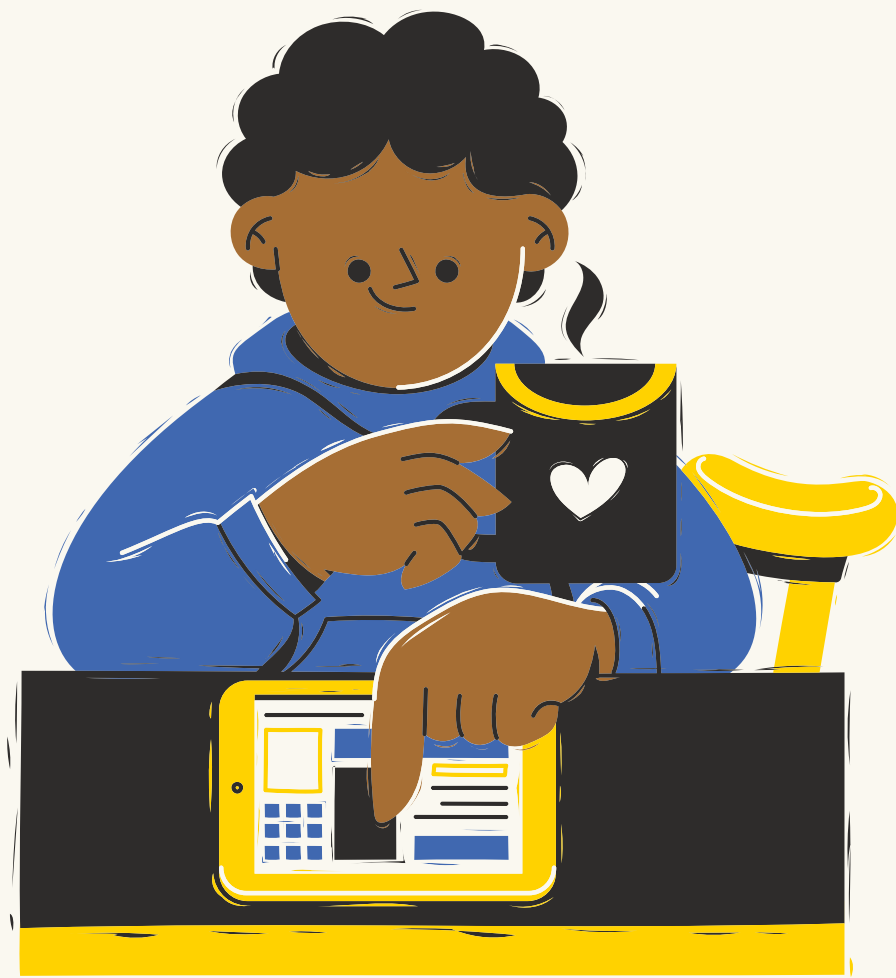
Wire.requestFrom(address,
quantity)



What is the main idea?

Requests data from an I2C device with a specified address and specifies how many bytes you expect to receive

Interrupt Functions



`interrupts()`

`noInterrupts()`

Math Functions



abs()

Returns the absolute
(magnitude) value of a
number x

Example

```
int absoluteValue = abs(-7)
```

Output - absoluteValue = 7

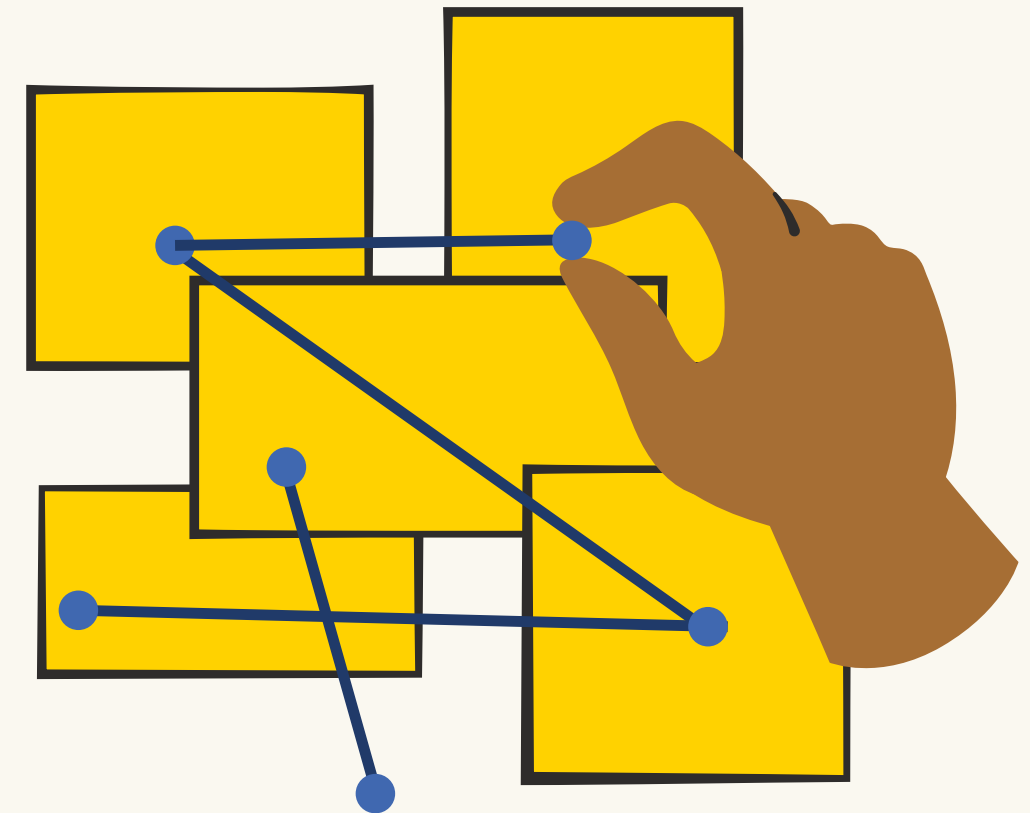
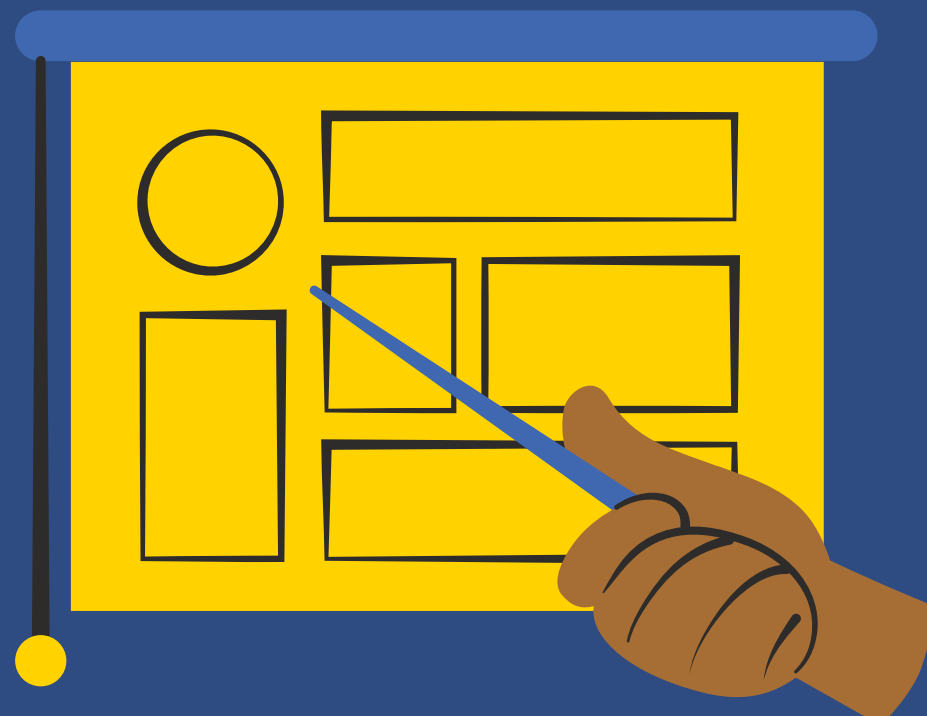


`constrain(x,a,b)`

restricts value x to be within the range defined by a and b .

If $x < a$ then $x = a$

if $x > b$ then $x = b$



Example -

```
int constrainValue = constrain  
(25, 10, 20);
```

Output - constrainValue = 20

Map()

Map(value, fromLow, fromHigh, toLow, toHigh)

Remaps a value for 1 range to another.

It takes a value within the range [fromLow, fromHigh] and maps it to the corresponding value within the range [toLow, toHigh]



Example

```
int mapValue = map(50, 0, 100, 0, 255)  
Output - mapValue = 127
```

Map Function Calculation

$$\text{mapValue} = \left(\frac{\text{value} - \text{fromLow}}{\text{fromHigh} - \text{fromLow}} \right) \times (\text{toHigh} - \text{toLow}) + \text{toLow}$$

```
mapValue = map(50, 0, 100, 0, 255);
```



Math Functions



`max (a, b)`

`min (a, b)`

`pow(base, exponent)`

`sq (x)`

`sqrt (x)`

FUNCTIONS

setup ()

Once

loop ()

Multiple Times

VALUES (VARIABLE AND CONSTANTS)

Variable

Data types

- int
- float
- char
- bool

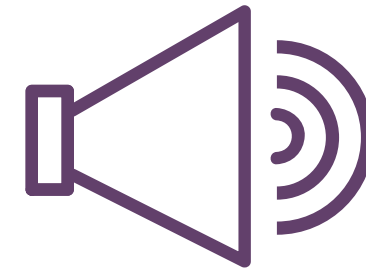
Constant

- **Predefined** - HIGH, LOW, INPUT, OUTPUT
- Using const keyword

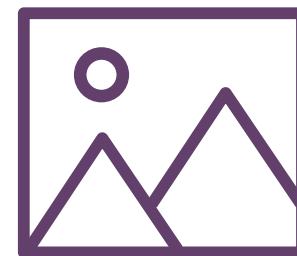
Structure



Conditional Statements



Loops



Switch Statements

CONDITIONAL STATEMENTS



01

if statement

02

if-else statement

Loops

1.

for

2.

while

3.

do-while

